Нам следует немного пересмотреть концепцию паралелльной lstm обработки.
Сейчас размер батча к примеру = 8, а вот hidden>100.
Поэтоиу нам следует по другому проводить вычислени я.

```
void atomic_add_float(global volatile float* source, const float val) {
    union { unsigned int intVal; float floatVal; } next, expected, current;
    current.floatVal = *source;
    do {
        expected.floatVal = current.floatVal;
        next.floatVal = expected.floatVal + val;
        current.intVal = atomic_cmpxchg((global volatile unsigned int*)source, expected.intVal,
next.intVal);
    } while (current.intVal != expected.intVal);
}

/**
 * Монолитное ядро прямого прохода LSTM.
 * Сетка запуска (Global Work Size): [BATCH_SIZE]
 */
kernel void lstm_forward(
    global const float* X,
    global const float* W,
    global const float* U,
    global const float* bias,
    global float* H_out,
    global float* workspace,
    // Временные глобальные буферы (выделяются в C#)
    global float* c_curr_buf,
    global float* h_curr_buf,
    const int batch_size,
    const int seq_len,
    const int in_features,
    const int hidden_size
) {
    int b = get_global_id(0) * get_global_size(1) + get_global_id(1);
    if (b >= batch_size) return;

    // Глобальные смещения текущего потока для h_t и c_t
    global float* c_curr = c_curr_buf + (b * hidden_size);
    global float* h_curr = h_curr_buf + (b * hidden_size);

    // Инициализируем начальные состояния нулями
    for (int h = 0; h < hidden_size; h++) {
        c_curr[h] = 0.0f;
        h_curr[h] = 0.0f;
    }

    int x_offset;
    int ws_offset;
    int h_out_offset;
    float val;

    for (int t = 0; t < seq_len; t++) {
```

```
        x_offset = b * (seq_len * in_features) + t * in_features;
        ws_offset = t * (batch_size * 4 * hidden_size) + b * (4 * hidden_size);
        h_out_offset = b * (seq_len * hidden_size) + t * hidden_size;

        for (int g = 0; g < 4 * hidden_size; g++) {
            val = bias[g];
            for (int i = 0; i < in_features; i++) {
                val += X[i + x_offset] * W[i * (4 * hidden_size) + g];
            }
            for (int h = 0; h < hidden_size; h++) {
                val += h_curr[h] * U[h * (4 * hidden_size) + g];
            }
            workspace[ws_offset + g] = val;
        }

        for (int h = 0; h < hidden_size; h++) {
            float raw_i = workspace[ws_offset + 0 * hidden_size + h];
            float raw_f = workspace[ws_offset + 1 * hidden_size + h];
            float raw_g = workspace[ws_offset + 2 * hidden_size + h];
            float raw_o = workspace[ws_offset + 3 * hidden_size + h];

            float i_gate = sigmoid(raw_i);
            float f_gate = sigmoid(raw_f);
            float g_gate = tanh(raw_g);
            float o_gate = sigmoid(raw_o);

            c_curr[h] = f_gate * c_curr[h] + i_gate * g_gate;
            h_curr[h] = o_gate * tanh(c_curr[h]);

            H_out[h_out_offset + h] = h_curr[h];
        }
    }
}

/**
 * Монолитное ядро обратного прохода LSTM (BPTT).
 * Сетка запуска (Global Work Size): [BATCH_SIZE]
 */
kernel void lstm_backward(
    global const float* X,
    global const float* W,
    global const float* U,
    global const float* H_out,
    global const float* workspace,
    global const float* grad_H_out,
    global float* grad_X,
    global float* grad_W,
    global float* grad_U,
    // Временные глобальные буферы (выделяются в C#)
    global float* dh_next_buf,
    global float* dc_next_buf,
    global float* c_accum_buf,
```

```
    global float* grad_gates_buf,
    const int batch_size,
    const int seq_len,
    const int in_features,
    const int hidden_size
) {
    int b = get_global_id(0) * get_global_size(1) + get_global_id(1);
    if (b >= batch_size) return;

    // Смещения под текущий батч-поток
    global float* dh_next = dh_next_buf + (b * hidden_size);
    global float* dc_next = dc_next_buf + (b * hidden_size);
    global float* c_accum = c_accum_buf + (b * hidden_size);
    global float* grad_gates = grad_gates_buf + (b * 4 * hidden_size); // У гейтов размерность в 4 раза больше!

    for (int h = 0; h < hidden_size; h++) {
        dh_next[h] = 0.0f;
        dc_next[h] = 0.0f;
        c_accum[h] = 0.0f;
    }

    for (int t = 0; t < seq_len; t++) {
        int ws_offset = t * (batch_size * 4 * hidden_size) + b * (4 * hidden_size);
        for (int h = 0; h < hidden_size; h++) {
            float i_gate = sigmoid(workspace[ws_offset + 0 * hidden_size + h]);
            float f_gate = sigmoid(workspace[ws_offset + 1 * hidden_size + h]);
            float g_gate = tanh(workspace[ws_offset + 2 * hidden_size + h]);
            c_accum[h] = f_gate * c_accum[h] + i_gate * g_gate;
        }
    }

    for (int t = seq_len - 1; t >= 0; t--) {
        int ws_offset = t * (batch_size * 4 * hidden_size) + b * (4 * hidden_size);
        int h_out_offset = b * (seq_len * hidden_size) + t * hidden_size;
        int x_offset = b * (seq_len * in_features) + t * in_features;

        // Вместо float c_prev[1024] используем регистр/переменную на лету
        for (int h = 0; h < hidden_size; h++) {
            float i_gate = sigmoid(workspace[ws_offset + 0 * hidden_size + h]);
            float f_gate = sigmoid(workspace[ws_offset + 1 * hidden_size + h]);
            float g_gate = tanh(workspace[ws_offset + 2 * hidden_size + h]);

            float c_prev_val = (f_gate != 0.0f) ? (c_accum[h] - i_gate * g_gate) / f_gate : 0.0f;

            float dh_curr = grad_H_out[h_out_offset + h] + dh_next[h];
            float raw_i = workspace[ws_offset + 0 * hidden_size + h];
            float raw_f = workspace[ws_offset + 1 * hidden_size + h];
            float raw_g = workspace[ws_offset + 2 * hidden_size + h];
            float raw_o = workspace[ws_offset + 3 * hidden_size + h];

            float o_gate = sigmoid(raw_o);
```

```
            float tanh_c = tanh(c_accum[h]);

            float d_raw_o = dh_curr * tanh_c * (o_gate * (1.0f - o_gate));
            float dc_curr = dh_curr * o_gate * (1.0f - tanh_c * tanh_c) + dc_next[h];

            float d_raw_f = dc_curr * c_prev_val * (f_gate * (1.0f - f_gate));
            float d_raw_i = dc_curr * g_gate * (i_gate * (1.0f - i_gate));
            float d_raw_g = dc_curr * i_gate * (1.0f - g_gate * g_gate);

            grad_gates[0 * hidden_size + h] = d_raw_i;
            grad_gates[1 * hidden_size + h] = d_raw_f;
            grad_gates[2 * hidden_size + h] = d_raw_g;
            grad_gates[3 * hidden_size + h] = d_raw_o;

            dc_next[h] = dc_curr * f_gate;
            c_accum[h] = c_prev_val; // Откатываем c_accum назад для следующей итерации t
        }

        int x_grad_offset = b * (seq_len * in_features) + t * in_features;
        for (int i = 0; i < in_features; i++) {
            float sum_x = 0.0f;
            for (int g = 0; g < 4 * hidden_size; g++) {
                sum_x += grad_gates[g] * W[i * (4 * hidden_size) + g];
            }
            grad_X[x_grad_offset + i] = sum_x;
        }

        for (int h = 0; h < hidden_size; h++) {
            float sum_h = 0.0f;
            for (int g = 0; g < 4 * hidden_size; g++) {
                sum_h += grad_gates[g] * U[h * (4 * hidden_size) + g];
            }
            dh_next[h] = sum_h;
        }

        int h_prev_offset = (t > 0) ? (b * (seq_len * hidden_size) + (t - 1) * hidden_size) : -1;
        for (int g = 0; g < 4 * hidden_size; g++) {
            float g_grad = grad_gates[g];
            for (int i = 0; i < in_features; i++) {
                float w_delta = X[x_offset + i] * g_grad;
                atomic_add_float(&grad_W[i * (4 * hidden_size) + g], w_delta);
            }
            if (h_prev_offset != -1) {
                for (int h = 0; h < hidden_size; h++) {
                    float u_delta = H_out[h_prev_offset + h] * g_grad;
                    atomic_add_float(&grad_U[h * (4 * hidden_size) + g], u_delta);
                }
            }
        }
    }
}
```

Чтобы ускорить код, мы проименим барьеры.
get_global_id (0) будет соотвевоть batch size
get_global_id(1) – hidden_size
Тогда на примере прямого прохода:
Псевдокод:
если get_globa_id(1)==0:

```
// Инициализируем начальные состояния нулями
for (int h = 0; h < hidden_size; h++) {
    c_curr[h] = 0.0f;
    h_curr[h] = 0.0f;
}
```

Дальше ставим барьер синхронизации потоков.
Затем ничинаем цикл по t:

```
46
47      int x_offset;
48      int ws_offset;
49      int h_out_offset;
50      float val;
51
52      for (int t = 0; t < seq_len; t++) {
53          x_offset = b * (seq_len * in_features) + t * in_features;
54          ws_offset = t * (batch_size * 4 * hidden_size) + b * (4 * hidden_size);
55          h_out_offset = b * (seq_len * hidden_size) + t * hidden_size;
56
```

Дальше нам нужно развернуть цикл

```
56
57          for (int g = 0; g < 4 * hidden_size; g++) {
58              val = bias[g];
59              for (int i = 0; i < in_features; i++) {
60                  val += X[i + x_offset] * W[i * (4 * hidden_size) + g];
61              }
62              for (int h = 0; h < hidden_size; h++) {
63                  val += h_curr[h] * U[h * (4 * hidden_size) + g];
64              }
65              workspace[ws_offset + g] = val;
66          }
67
```

чтобы он принял вид примерно:
gid=get_global_id(1);
for(int g=0;g<4;g++){
val = bias[hidden_size*g+gid];
/*дальше прнибавлем x вектор*/
/*затем прибавляем 2 цикл (обычный последовательный проход, g заменяеться на hidden_size*g+gid и записываем в workspace .

Послен окончания цикла по g выполнем синхронизацию (барьер).

```
for (int h = 0; h < hidden_size; h++) {
    float raw_i = workspace[ws_offset + 0 * hidden_size + h];
    float raw_f = workspace[ws_offset + 1 * hidden_size + h];
    float raw_g = workspace[ws_offset + 2 * hidden_size + h];
    float raw_o = workspace[ws_offset + 3 * hidden_size + h];

    float i_gate = sigmoid(raw_i);
    float f_gate = sigmoid(raw_f);
    float g_gate = tanh(raw_g);
    float o_gate = sigmoid(raw_o);

    c_curr[h] = f_gate * c_curr[h] + i_gate * g_gate;
    h_curr[h] = o_gate * tanh(c_curr[h]);

    H_out[h_out_offset + h] = h_curr[h];
}
```

Этот цикл превращаеться в набор инструкций вида :
`float raw_i = workspace[ws_offset + 0 * hidden_size + get_global_id(1)];`

после строки H_out соотвенно тоже идет синхронизация.

Обратный проход перестраиваеться анлогичным образом.